

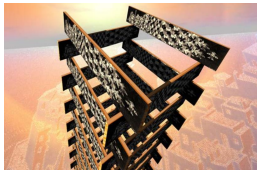
Analyses de pointeurs et de tableaux

TAS : Typage et analyse statique
M2, Master STL, Sorbonne Université

Antoine Miné

Année 2024–2025

Cours 13
17 janvier 2025



Analyses de pointeurs

Introduction

Nous avons considéré jusqu'à présent uniquement des **analyses numériques**.

Les langages réalistes ont :

- des structures et des tableaux ;
- des **références** (vision haut niveau : Java, OCaml) ;
- et de l'**allocation dynamique** (`malloc`, `new`) ;
- voire des **pointeurs** avec de l'**arithmétique de pointeurs**.
(vision bas niveau : C, C++, binaires)

Une analyse doit tenir compte des références et des pointeurs car :

- 1 ils causent des **erreurs spécifiques** ;
accès à une référence NULL, à un pointeur invalide, débordement de mémoire, double libération, etc.
⇒ nous voulons vérifier statiquement ces erreurs ;
- 2 leur présence rend l'analyse des propriétés numériques **plus complexe**.
simplement les ignorer donne des résultats non sûrs !

Exemple 1 : analyse de constantes avec pointeurs

Exemple

```
int x = 2;  
int *p = &x;  
(*p)++;  
int y = x + 1;
```

Une analyse de constantes qui ignore les pointeurs trouverait :
x constant égal à 2.

Un compilateur optimisant utilisant cette analyse de constantes
remplacerait `y = x + 1` par `y = 3`
ce qui est **faux** !

Problème : un accès indirect par pointeur modifie la valeur de x.
C'est le problème de l'**aliasing**.

Exemple 2 : analyse de manipulations mémoire en C

Exemple

```
char* p = malloc(100);  
char* q = p;  
for (int i=0; i<10; i++, q++) *q = 12;  
free(p);  
(*q)++;
```

- après malloc, p pointe vers un bloc valide;
- après q = p, q pointe vers un bloc valide;
- free(p) libère le bloc
⇒ p et q deviennent invalides.
- (*q)++ provoque une erreur !

Une analyse statique de pointeurs permet de détecter ces erreurs.

Exemple 3 : analyse de classe en Java

Exemple

```
class Rectangle {  
    public void draw() { ...}  
}  
  
class Square extends Rectangle {  
    @Override public void draw() { ...}  
}  
  
Rectangle x;  
x = new Square();  
...  
x.draw();
```

L'analyse statique permet une **optimisation** à la compilation :

- x contient une référence vers un objet de classe Square ;
- $\implies$ x.draw peut être compilé en un **appel direct** à la méthode de Square.
(au lieu d'un appel indirect dans une table de méthodes, plus coûteux)

Analyse de pointeurs et analyse d'alias

Deux problèmes liés :

- **analyse de pointeurs** :

étant donné un pointeur p :

- vers quelles variables p peut-il pointer ?
- vers quels blocs alloués dynamiquement p peut-il pointer ?

- **analyse d'alias** :

étant donnés deux pointeurs p et q :

- p et q peuvent-ils pointer vers le même bloc mémoire ?

L'analyse d'alias peut se réduire à l'analyse de pointeurs.

Note : il existe également des analyses d'alias dédiées, mais nous n'en parlerons pas.

Plan

But : montrer des exemples

- 1 d'analyses **non numériques** : les **pointeurs** ;
- 2 d'analyses **insensibles au flot de contrôle**, très efficaces
⇒ utilisables dans un compilateur.
(mais moins précises que les analyses sensibles au flot de contrôle vues jusqu'à présent)

Plan :

- langage de pointeurs simple ;
- sémantique **concrète**, non calculable ;
- abstraction par **site d'allocation** ;
- **analyse** par interprétation abstraite, sensible au flot de contrôle ;
- **analyses** insensibles au flot de contrôle :
 - analyse d'**Andersen** ;
 - analyse de **Steensgaard**.

Langage

Syntaxe

Instructions

<i>stat</i>	::=	$X \leftarrow \text{alloc}()$	(allocation)
		$X \leftarrow \&Y$	(prise de référence)
		$X \leftarrow *Y$	(lecture par référence)
		$*X \leftarrow Y$	(écriture par référence)
		$X \leftarrow \text{expr}$	(affectation numérique)
		...	

- nous ajoutons aux instructions numériques des **instructions de pointeur** ;
- une variable peut contenir un **pointeur** ou un **entier** ;
- les manipulations de pointeurs sont réduites à des instructions atomiques ;
nous supposons que les instructions complexes sont décomposées en instructions atomiques ;
ainsi, $*p = *q + 1$ s'écrit : $\text{tmp} \leftarrow *q$; $\text{tmp} \leftarrow \text{tmp} + 1$; $*p \leftarrow \text{tmp}$
- **alloc()** **alloue** une case mémoire pouvant contenir un pointeur ou un entier et retourne son adresse ;
- pas d'arithmétique de pointeur, pas de libération de mémoire.

Modélisation concrète de la mémoire

L'ajout de pointeurs nécessite d'enrichir la notion de **variable** et de **valeur**.

Adresses $\mathbb{A}$: $\mathbb{A} \stackrel{\text{def}}{=} \mathbb{V} \cup \mathbb{H}$

Dénote une portion de mémoire où une valeur peut être stockée.

Une adresse peut être dans :

- $\mathbb{V}$: les **variables** du programme (comme avant)
- $\mathbb{H}$: **le tas**, un ensemble **infini** d'adresses mémoires
alloc() pioche une nouvelle adresse non utilisée dans le tas $\mathbb{H}$

Valeurs $\mathbb{V}$: $\mathbb{V} \stackrel{\text{def}}{=} \mathbb{A} \cup \mathbb{Z}$

- l'**adresse** d'une **variable** $\mathbb{V} \subseteq \mathbb{A}$
- ou l'**adresse** d'un **bloc alloué** dynamiquement $\mathbb{H} \subseteq \mathbb{A}$
- ou un **entier**, dans $\mathbb{Z}$

Modélisation concrète de la mémoire (suite)

Environnements concrets : $\mathcal{E} \stackrel{\text{def}}{=} \mathbb{A} \rightarrow \mathcal{V}$

- associe une **valeur** à une **adresse mémoire**
- $\rightarrow$ dénote une **fonction partielle**
toutes les adresses ne sont pas allouées ! (seulement un nombre fini)
- si $\rho \in \mathcal{E}$, $\text{dom}(\rho) \subseteq \mathbb{A}$ dénote les adresses effectivement allouées
 - $\rho(v)$ est indéfini si $v \notin \text{dom}(\rho)$
 - à chaque instant, $\text{dom}(\rho)$ est un ensemble fini
mais $\mathbb{A}$ est infini, pour modéliser des programmes utilisant une mémoire arbitraire
 - $\mathbb{V} \subseteq \text{dom}(\rho)$: les variables existent toujours en mémoire

Sémantique concrète

$S[\![\text{stat}]\!] : \mathcal{P}(\mathcal{E}) \rightarrow \mathcal{P}(\mathcal{E})$

$S[\![X \leftarrow \text{alloc}()]\!] R$	$\stackrel{\text{def}}{=}$	$\{ \rho[X \mapsto \ell, \ell \mapsto 0] \mid \rho \in R, \ell \notin \text{dom}(\rho) \}$
$S[\![X \leftarrow \&Y]\!] R$	$\stackrel{\text{def}}{=}$	$\{ \rho[X \mapsto Y] \mid \rho \in R \}$
$S[\![X \leftarrow Y]\!] R$	$\stackrel{\text{def}}{=}$	$\{ \rho[X \mapsto \rho(Y)] \mid \rho \in R \}$
$S[\![X \leftarrow *Y]\!] R$	$\stackrel{\text{def}}{=}$	$\{ \rho[X \mapsto \rho(\rho(Y))] \mid \rho \in R, \rho(Y) \in \text{dom}(\rho) \}$
$S[\![*X \leftarrow Y]\!] R$	$\stackrel{\text{def}}{=}$	$\{ \rho[\rho(X) \mapsto \rho(Y)] \mid \rho \in R, \rho(X) \in \text{dom}(\rho) \}$

Notes :

- $\rho[\ell \mapsto x], \ell \notin \text{dom}(\rho)$ enrichit $\text{dom}(\rho)$
- les blocs alloués sont initialisés à 0
- $X \mapsto Y$ stocke l'adresse de la variable Y dans la variable X
- $X \mapsto \rho(Y)$ stocke la valeur de la variable Y dans la variable X
- $X \mapsto \rho(\rho(Y))$ stocke la valeur pointée par Y dans la variable X
- $\rho(X) \mapsto v$ stocke v à l'adresse pointée par la variable X
- $\rho(Y) \in \text{dom}(\rho)$ signifie : Y pointe sur une adresse valide
- pas de modification à $S[\![X \leftarrow \text{expr}]\!]$, $C[\![\cdot]\!]$

car les expressions ne contiennent que des variables,

il faut vérifier que $\rho(V) \in \mathbb{Z}$ pour les variables V des expressions, i.e., on ne manipule pas d'adresse

Analyse statique

Principe d'analyse

Principe : décomposition du problème

1 analyse de pointeurs : $ptr : \mathbb{V} \rightarrow \mathcal{P}(\mathbb{A})$

$\implies$ détermine où chaque variable peut pointer

2 utiliser l'information de l'analyse pour **éliminer** les pointeurs du programme par **transformation** de programme

Exemple :

<code>int x = 2;</code>		<code>int x = 2;</code>
<code>int *p = &x;</code>	$ptr(p) = \{x\}$	
<code>(*p)++;</code>	$\implies$	<code>x++;</code>
<code>int y = x + 1;</code>		<code>int y = x + 1;</code>

3 analyse numérique : dans $\mathcal{P}(\mathbb{A} \rightarrow \mathbb{Z})$

réutilisation des domaines numériques présentés en cours

Note :

une analyse combinée de pointeurs et d'entiers est possible, avec un **produit réduit** ; elle serait plus précise, mais un peu plus complexe à mettre en œuvre.

Difficulté : affectation faible

Exemple

```

X ← 1; Y ← 2;
if ... then P ← &X else P ← &Y;
Z ← *P;
*P ← 3

```

Une analyse de pointeurs donnerait $ptr(P) = \{X, Y\}$.

Comment interpréter $Z \leftarrow *P$ et $*P \leftarrow 3$?

Indétermination sur $*P \implies$ il faut traiter **tous les cas** !

Exemple : dans les domaines non relationnels (e.g., intervalles)

$$\blacksquare S^\# \llbracket Z \leftarrow \{X, Y\} \rrbracket R^\# \stackrel{\text{def}}{=} R^\# [Z \mapsto R^\#(X) \cup^\# R^\#(Y)]$$

union abstraite $\cup^\#$ de toutes les variables lues

Difficulté : affectation faible

Exemple

```

X ← 1; Y ← 2;
if ... then P ← &X else P ← &Y;
Z ← *P;
*P ← 3

```

Une analyse de pointeurs donnerait $ptr(P) = \{X, Y\}$.

Comment interpréter $Z \leftarrow *P$ et $*P \leftarrow 3$?

Indétermination sur $*P \implies$ il faut traiter **tous les cas** !

Exemple : dans les domaines non relationnels (e.g., intervalles)

- $S^\sharp \llbracket \{X, Y\} \leftarrow e \rrbracket R^\sharp \stackrel{\text{def}}{=} R^\sharp[X \mapsto R^\sharp(X) \cup^\sharp E^\sharp \llbracket e \rrbracket R^\sharp, Y \mapsto R^\sharp(Y) \cup^\sharp E^\sharp \llbracket e \rrbracket R^\sharp]$
 - toutes les variables de $ptr(P)$ sont mises à jour ;
 - une variable peut retenir sa valeur précédente
($V \mapsto R^\sharp(V) \cup^\sharp \dots$)

$\implies$ les intervalles donnent : $Z \in [1, 2]$, $X \in [1, 3]$, $Y \in [2, 3]$.

Difficulté : allocation non bornée

Exemple

```
 $X \leftarrow \text{rand}(0, +\infty);$   
while  $i < X$  do  
     $P \leftarrow \text{alloc}();$   
     $*P \leftarrow i$   
     $i \leftarrow i + 1$   
done
```

Rappel : $\mathbb{A}$ est infini.

nécessaire pour donner un sens à un programme avec allocation non bornée

$\implies$ **$\text{ptr}(P)$ peut être infini !** (union des cases allouées à tous les tours de boucle)

or, l'analyse numérique ne peut se faire que sur un nombre fini de variables. . .

Solution : abstraction par site d'allocation

Exemple

```

X ← rand(0, +∞);
while i < X do
    P ← allocℓ();
    *P ← i
    i ← i + 1
done

```

Solution : abstraire $\mathbb{A}$ en un ensemble fini $\mathbb{A}^\#$ d'adresses abstraites

$\mathbb{A}^\#$ contient :

- les variables $\mathbb{V}$
- une adresse abstraite $\ell^\#$ pour chaque instruction **alloc**() du programme
 ℓ est un point de programme syntaxique (position d'une instruction **alloc**)
 $\ell^\# \in \mathbb{A}^\#$ représente l'ensemble de tous les blocs alloués à ℓ par **alloc**^ℓ()

Exemple : $\mathbb{A}^\# = \{X, i, P, \ell^\#\}, ptr(P) = \{\ell^\#\}$

Avantage : $\mathbb{A}^\#$ est maintenant fini

Inconvénient : tout accès à $\ell^\#$ génère une affectation faible (perte de précision)

Analyse abstraite de pointeurs

Domaine abstrait : $\mathcal{E}^\# \stackrel{\text{def}}{=} \mathbb{A}^\# \rightarrow \mathcal{P}(\mathbb{A}^\#)$

- utilisation des adresses abstraites : $\mathbb{A}^\#$
- inférence des informations de pointeur uniquement : $\mathcal{P}(\mathbb{A}^\#)$
pas de propriété numérique !

$S^\# \llbracket \text{stat} \rrbracket : \mathcal{E}^\# \rightarrow \mathcal{E}^\#$

$S^\# \llbracket X \leftarrow \text{alloc}^\ell() \rrbracket R^\# \stackrel{\text{def}}{=} R^\# [X \mapsto \{\ell^\#\}]$

$S^\# \llbracket X \leftarrow \&Y \rrbracket R^\# \stackrel{\text{def}}{=} R^\# [X \mapsto \{Y\}]$

$S^\# \llbracket X \leftarrow Y \rrbracket R^\# \stackrel{\text{def}}{=} R^\# [X \mapsto R^\#(Y)]$

$S^\# \llbracket X \leftarrow *Y \rrbracket R^\# \stackrel{\text{def}}{=} R^\# [X \mapsto \bigcup_{v \in R^\#(Y)} R^\#(v)]$

$S^\# \llbracket *X \leftarrow Y \rrbracket R^\# \stackrel{\text{def}}{=} \begin{cases} R^\# [V \mapsto R^\#(Y)] & \text{si } R^\#(X) = \{V\} \text{ et } V \in \mathbb{V} \\ R^\# [\forall p \in R^\#(X): p \mapsto R^\#(p) \cup R^\#(Y)] & \text{sinon} \quad (\text{affectation faible}) \end{cases}$

Sémantique abstraite des manipulations de pointeurs :

- **analyse non relationnelle** : chaque variable est abstraite indépendamment ;
- utilisation d'**affectations faibles** en cas d'indétermination sur la destination.

Analyse abstraite de pointeurs (suite)

$S^\# \llbracket \text{stat} \rrbracket : \mathcal{E}^\# \rightarrow \mathcal{E}^\#$

$S^\# \llbracket s_1; s_2 \rrbracket R^\#$	$\stackrel{\text{def}}{=}$	$S^\# \llbracket s_2 \rrbracket (S^\# \llbracket s_1 \rrbracket R^\#)$
$S^\# \llbracket X \leftarrow \text{expr} \rrbracket R^\#$	$\stackrel{\text{def}}{=}$	$R^\# [X \mapsto \emptyset]$
$S^\# \llbracket \text{if } c \text{ then } s_1 \text{ else } s_2 \rrbracket R^\#$	$\stackrel{\text{def}}{=}$	$(S^\# \llbracket s_1 \rrbracket R^\#) \dot{\cup} (S^\# \llbracket s_2 \rrbracket R^\#)$
$S^\# \llbracket \text{while } c \text{ do } s \rrbracket R^\#$	$\stackrel{\text{def}}{=}$	$\lim \lambda X^\#. (R^\# \dot{\cup} S^\# \llbracket s \rrbracket X^\#)$

Sémantique abstraite du fragment sans pointeur :

- interprétation par induction sur la syntaxe ;
- $\dot{\cup}$ est l'**union point à point** : $\forall A \in \mathbb{A}^\# : (R^\# \dot{\cup} S^\#)(A) = R^\#(A) \cup S^\#(A)$;
- les **affectations numériques** $X \leftarrow \text{expr}$ ne génèrent **pas de pointeur** ;
- les **tests numériques** sont interprétés comme des **choix non déterministes** ;
- $\mathcal{E}^\#$ est fini et a donc une **hauteur finie**
 $\implies$ les boucles calculent des itérations abstraites **sans élargissement**.

Analyses insensibles au flot de contrôle

Principe

Toutes les analyses vues en cours sont **sensibles au flot de contrôle** :

- elle **distinguent** l'état abstrait en fonction du point de programme
⇒ précis
- elle procèdent par **induction sur la syntaxe** du programme
en itérant les boucles
⇒ coûteux

Certaines applications, comme la compilation optimisée, nécessitent des analyses moins coûteuses (même si elles sont moins précises).

Analyses insensibles au flot de contrôle :

- manipulent un **unique environnement abstrait** global valable en tout point du programme ;
- permettent un **ordre arbitraire** de traitement des instructions du programme (l'analyse ne suit pas forcément le flot de contrôle)

Analyse d'Andersen

Principe :

- **associer** à chaque variable un ensemble d'adresses abstraites, $ptr : \mathbb{V} \rightarrow \mathbb{A}^\#$, à déterminer
- **extraire** de chaque **affectation** une **contrainte d'inclusion sur ptr**
- **résoudre** l'ensemble des contraintes extraites pour inférer ptr

l'instruction : est représentée par la contrainte :

$X \leftarrow \text{alloc}^\ell()$ $\{\ell\} \subseteq ptr(X)$

$X \leftarrow \&Y$ $\{Y\} \subseteq ptr(X)$

$X \leftarrow Y$ $ptr(Y) \subseteq ptr(X)$

$X \leftarrow *Y$ $\forall v \in ptr(Y): ptr(v) \subseteq ptr(X)$

$*X \leftarrow Y$ $\forall v \in ptr(X): ptr(Y) \subseteq ptr(v)$

- les constructions de test et de boucle sont ignorées
seules les affectations contenues dans ces construction sont extraites
- mélange de contraintes d'**inclusion** simples (faciles à traiter)
et de contraintes **quantifiées** (plus complexes)

Résolution des contraintes d'inclusion simples

Exemple :

programme :

$P \leftarrow \&X;$

$Q \leftarrow P;$

$P \leftarrow \mathbf{alloc}^\ell();$

$R \leftarrow P$

contraintes :

$\{X\} \subseteq ptr(P)$

$ptr(P) \subseteq ptr(Q)$

$\{\ell\} \subseteq ptr(P)$

$ptr(P) \subseteq ptr(R)$

La **plus petite solution** est : $ptr(P) = ptr(Q) = ptr(R) = \{X, \ell\}$.

Note : perte de précision, en réalité Q ne peut pas pointer sur ℓ , et R ne peut pas pointer sur X

La résolution peut se faire par un algorithme de **graphe** :

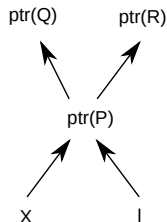
- un nœud pour chaque adresse $a \in \mathbb{A}^\sharp$, représentant l'ensemble $\{a\}$;
- un nœud pour chaque pointeur à déterminer : $ptr(P)$;
- un arc $v \rightarrow w$ pour chaque contrainte $v \subseteq w$;
- calcul de la **clôture transitive**

$ptr(P) = \{a \in \mathbb{A}^\sharp \mid \text{la clôture transitive a un arc } a \rightarrow ptr(P)\}$

$\implies$ coût cubique

Exemple de résolution

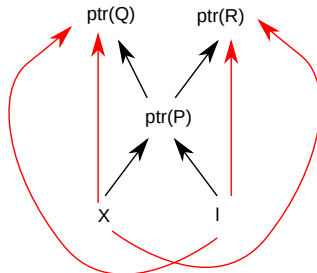
$\{X\} \subseteq ptr(P)$
 $ptr(P) \subseteq ptr(Q)$
 $\{\ell\} \subseteq ptr(P)$
 $ptr(P) \subseteq ptr(R)$



Système initial.

Exemple de résolution

$$\begin{aligned}\{X\} &\subseteq ptr(P) \\ ptr(P) &\subseteq ptr(Q) \\ \{\ell\} &\subseteq ptr(P) \\ ptr(P) &\subseteq ptr(R)\end{aligned}$$



Après la clôture transitive.

$$ptr(P) = ptr(Q) = ptr(R) = \{X, \ell\}$$

Résolution des contraintes complexes

<u>Exemple :</u>	programme :	contraintes :
	$P \leftarrow \&X;$	$\{X\} \subseteq ptr(P)$
	$Q \leftarrow P;$	$ptr(P) \subseteq ptr(Q)$
	$*P \leftarrow Q;$	$\forall v \in ptr(P): ptr(Q) \subseteq ptr(v)$
	$P \leftarrow \&Y;$	$\{Y\} \subseteq ptr(P)$
	$S \leftarrow *P;$	$\forall v \in ptr(P): ptr(v) \subseteq ptr(S)$

Principe de résolution :

- 1 création d'un graphe pour les contraintes simples d'inclusion
- 2 calcul de la clôture transitive
donnant une première approximation de ptr
- 3 ajout d'arcs $v \rightarrow w$ par **interprétation** des contraintes complexes
e.g., substituer la valeur trouvée de $ptr(P)$ dans $\forall v \in ptr(P): ptr(Q) \subseteq ptr(v)$
- 4 reprendre à l'étape 2, et itérer 2 et 3 jusqu'à stabilisation

Le coût est toujours cubique.

Pour notre exemple, nous trouvons :

$ptr(P) = ptr(Q) = ptr(S) = ptr(X) = ptr(Y) = \{X, Y\}$ (cycle sur X et Y)

Exemple de résolution

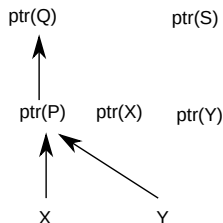
$$\{X\} \subseteq ptr(P)$$

$$\{Y\} \subseteq ptr(P)$$

$$ptr(P) \subseteq ptr(Q)$$

$$\forall v \in ptr(P): ptr(Q) \subseteq ptr(v)$$

$$\forall v \in ptr(P): ptr(v) \subseteq ptr(S)$$



Système initial.

Exemple de résolution

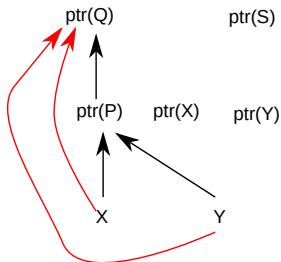
$$\{X\} \subseteq ptr(P)$$

$$\{Y\} \subseteq ptr(P)$$

$$ptr(P) \subseteq ptr(Q)$$

$$\forall v \in ptr(P): ptr(Q) \subseteq ptr(v)$$

$$\forall v \in ptr(P): ptr(v) \subseteq ptr(S)$$



Après une clôture transitive.

Exemple de résolution

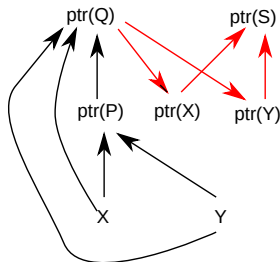
$$\{X\} \subseteq ptr(P)$$

$$\{Y\} \subseteq ptr(P)$$

$$ptr(P) \subseteq ptr(Q)$$

$$\forall v \in ptr(P): ptr(Q) \subseteq ptr(v)$$

$$\forall v \in ptr(P): ptr(v) \subseteq ptr(S)$$



Après ajout des arcs des quantificateurs

$$\forall v \in ptr(P) = \{X, Y\}.$$

Exemple de résolution

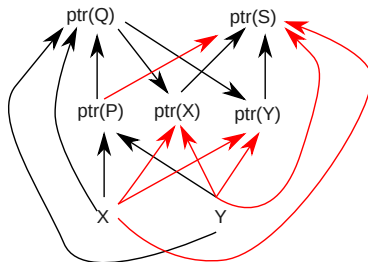
$$\{X\} \subseteq ptr(P)$$

$$\{Y\} \subseteq ptr(P)$$

$$ptr(P) \subseteq ptr(Q)$$

$$\forall v \in ptr(P): ptr(Q) \subseteq ptr(v)$$

$$\forall v \in ptr(P): ptr(v) \subseteq ptr(S)$$



Après une deuxième clôture transitive

$$ptr(P) = ptr(Q) = ptr(S) = ptr(X) = ptr(Y) = \{X, Y\}.$$

Analyse de Steensgaard

Principe :

Remplacer les contraintes d'**inclusion** entre deux ensembles $ptr(X)$ par des contraintes d'**égalité**.

(nous gardons les contraintes d'inclusion de la forme $\{X\} \subseteq ptr(Y)$)

l'instruction : est représentée par la contrainte :

$X \leftarrow Y$	$ptr(Y) = ptr(X)$
$X \leftarrow *Y$	$\forall v \in ptr(Y): ptr(v) = ptr(X)$
$*X \leftarrow Y$	$\forall v \in ptr(X): ptr(Y) = ptr(v)$
$X \leftarrow \text{alloc}^\ell()$	$\{\ell\} \subseteq ptr(X)$
$X \leftarrow \&Y$	$\{Y\} \subseteq ptr(X)$

La clôture transitive d'une relation d'**égalité** est bien **plus efficace** à calculer que la clôture transitive d'une relation d'inclusion !

coût quasi-linéaire avec une structure de données de type *union-find*

Mais l'analyse est moins précise.

Précision des analyses de Steensgaard et d'Andersen

Exemple :

programme :

```
P ← &X;
Q ← &Y;
Q ← P
```

Andersen :

$$\begin{aligned}\{X\} &\subseteq ptr(P) \\ \{Y\} &\subseteq ptr(Q) \\ ptr(P) &\subseteq ptr(Q)\end{aligned}$$

Steensgaard :

$$\begin{aligned}\{X\} &\subseteq ptr(P) \\ \{Y\} &\subseteq ptr(Q) \\ ptr(P) &= ptr(Q)\end{aligned}$$

L'analyse d'Andersen trouve : $ptr(P) = \{X\}$, $ptr(Q) = \{X, Y\}$.

L'analyse de Steensgaard trouve : $ptr(P) = \{X, Y\}$, $ptr(Q) = \{X, Y\}$.
Elle est moins précise.

Niveaux de sensibilité

Outre le choix entre sensibilité et insensibilité au flot de contrôle, plusieurs autres choix permettent de contrôler le rapport coût / précision :

- **sensibilité au contexte d'appel**

en présence de procédures, distinguer les sites d'appel, traiter les blocs alloués comme des variables abstraites distinctes dans $\mathbb{A}^\#$

Exemple :

```
void* f() { return malloc(100); }
```

```
void main() { a = f(); b = f(); }
```

les blocs a et b seront abstraits séparément, car les deux appels à f sont distingués

- **sensibilité au champ**

en présence de structures, chaque champ a une variable abstraite distincte dans $\mathbb{A}^\#$ plus précis et plus coûteux qu'avec une seule variable abstraite par structure (représentant tous ses champs, avec affectation faible)

- des variantes **mélangeant** Steensgaard et Andersen existent également

- contraintes d'inclusion pour les pointeurs de surface

variables qui ne sont pas référencées par d'autres pointeurs

- contraintes d'égalité pour les pointeurs profonds

traitement précis du passage par référence sans compromettre l'efficacité

Abstractions de tableaux

Exemple

Exemple : séquence croissante

```

 $p[0] \leftarrow 0; B[0] \leftarrow A[0];$ 
 $i \leftarrow 1; k \leftarrow 1;$ 
while  $i < N$  do
    if  $A[i] > B[k - 1]$  then
         $B[k] \leftarrow A[i];$ 
         $p[k] \leftarrow i;$ 
         $k \leftarrow k + 1;$ 
     $i \leftarrow i + 1$ 
done

```

Étant donné un tableau $A[0], \dots, A[N - 1]$
 le programme extrait un sous-tableau croissant $B[0], \dots, B[k - 1]$
 et les indices correspondants $p[0], \dots, p[k - 1]$

Invariants :

$$1 \leq k \leq i \leq N \quad \forall x < k: B[x] = A[p[x]]$$

$$\forall x: 0 \leq p[x] < N \quad \forall x < k - 1: B[x + 1] > B[x]$$

Plan

- Syntaxe et sémantique concrète
- Sémantique abstraite **non relationnelle**
e.g., $\forall i: A[i] \leq \text{constante}$
 - application à l'analyse d'intervalles
- Sémantique abstraite **relationnelle** (mais uniforme)
e.g., $\forall i: A[i] \leq V$
 - opérations *expand* et *fold*
 - application à l'analyse dans les polyèdres
- Un mot sur les abstractions **non uniformes**
e.g., $\forall i: A[i] \leq i$

Extension de la syntaxe

Nouvelles expressions et instructions

$expr$	$::=$	V	accès scalaire, $V \in \mathbb{V}$
		$A[expr]$	accès de tableau, $A \in \mathbb{A}$
		$\dots$	
$stat$	$::=$	$V \leftarrow expr$	affectation scalaire, $V \in \mathbb{V}$
		$A[expr] \leftarrow expr$	affectation de tableau, $A \in \mathbb{A}$
		$\dots$	

Le langage a maintenant deux manières d'accéder à la mémoire :

- $\mathbb{V}$: variables scalaires entières (comme avant)
- $\mathbb{A}$: des **tableaux** d'entiers (nouveaux)
 - les tableaux sont indicés par des **entiers positifs**
 - les tableaux sont supposés **non bornés**
(pour simplifier, nous ignorons les dépassements)

$\Rightarrow$ un tableau A est similaire à une fonction ou une **table** $A : \mathbb{N} \rightarrow \mathbb{Z}$

Sémantique concrète

Environnements concrets : $\mathcal{E} \stackrel{\text{def}}{=} (\mathbb{V} \cup (\mathbb{A} \times \mathbb{N})) \rightarrow \mathbb{Z}$

$\rho \in \mathcal{E}$ affecte une valeur entière à chaque « cellule mémoire » :

- $\rho(V)$ pour chaque variable scalaire $V \in \mathbb{V}$ et
- $\rho(\langle A, i \rangle)$ pour chaque case d'indice i du tableau $A \in \mathbb{A}$, $i \geq 0$

Sémantique concrète :

$$\begin{aligned}
 E[V] \rho & \stackrel{\text{def}}{=} \{\rho(V)\} \\
 E[A[e]] \rho & \stackrel{\text{def}}{=} \{\rho(\langle A, i \rangle) \mid i \in E[e] \rho\} \\
 S[V \leftarrow e] R & \stackrel{\text{def}}{=} \{\rho[V \mapsto v] \mid \rho \in R, v \in E[e] \rho\} \\
 S[A[f] \leftarrow e] R & \stackrel{\text{def}}{=} \{\rho[\langle A, i \rangle \mapsto v] \mid \rho \in R, v \in E[e] \rho, i \in E[f] \rho, i \geq 0\} \\
 \dots
 \end{aligned}$$

Abstractions non relationnelles

Abstraction de repliage

But : réutiliser les domaines numériques abstraits existants

Problème : les domaines numériques n'abstraient que des sous-ensembles de $\mathbb{Z}^n$, pour n fini

Solution : réduire $\mathcal{E}$ à des fonctions sur des ensembles **finis** de **variables abstraites**

Variables abstraites : $\mathbb{V}^\# \stackrel{\text{def}}{=} \mathbb{V} \cup \mathbb{A}$

- les variables scalaires de $\mathbb{V}$ sont exactement représentées dans $\mathbb{V}^\#$
- le contenu d'un **tableau** $A \in \mathbb{A}$ est abstrait par une unique **variable de résumé** $A^\#$ qui modélise le contenu de tout le tableau
- $\mathbb{V}^\#$ est **fini**

Correspondance de Galois de repliage :

$$(\mathcal{P}(\mathcal{E}), \subseteq) \xleftrightarrow[\alpha_s]{\gamma_s} (\mathcal{P}(\mathbb{V}^\# \rightarrow \mathbb{Z}), \subseteq)$$

- $\alpha_s(R) \stackrel{\text{def}}{=} \{ [V \mapsto \rho(V), A^\# \mapsto \rho(\langle A, \iota(A) \rangle)] \mid \rho \in R, \iota \in \mathbb{A} \rightarrow \mathbb{N} \}$
replie tous les éléments de tableau $\langle A, i \rangle$ dans la variable abstraite $A^\#$
- $\gamma_s(S) \stackrel{\text{def}}{=} \{ \rho \mid \forall \iota \in \mathbb{A} \rightarrow \mathbb{N} : [V \mapsto \rho(V), A^\# \mapsto \rho(\langle A, \iota(A) \rangle)] \in S \}$
on a bien : $\gamma_s(S) = \{ \rho \mid \alpha_s(\{\rho\}) \subseteq S \} = \cup \{ R \mid \alpha_s(R) \subseteq S \}$

Abstractions non relationnelles

Rappel : abstraction d'intervalle

- $\mathcal{P}(\mathbb{V}^\# \rightarrow \mathbb{Z})$ est abstrait en $\mathbb{V}^\# \rightarrow \mathcal{P}(\mathbb{Z})$ (abstraction Cartésienne)
- $\mathcal{P}(\mathbb{Z})$ est abstrait en un intervalle, dans $\mathbb{I}$

Note : l'abstraction Cartésienne et l'abstraction de repliage commutent

Sémantique abstraite : in $\mathcal{E}^\# \stackrel{\text{def}}{=} \mathbb{V}^\# \rightarrow \mathbb{I}$

- $E^\# \llbracket V \rrbracket X^\# \stackrel{\text{def}}{=} X^\#(V)$
 $E^\# \llbracket A[e] \rrbracket X^\# \stackrel{\text{def}}{=} X^\#(A^\#)$ (e est ignoré)
- $S^\# \llbracket V \leftarrow e \rrbracket X^\# \stackrel{\text{def}}{=} X^\# [V \mapsto E^\# \llbracket e \rrbracket X^\#]$
 $S^\# \llbracket A[f] \leftarrow e \rrbracket X^\# \stackrel{\text{def}}{=} X^\# [A^\# \mapsto X^\#(A^\#) \cup^\# E^\# \llbracket e \rrbracket X^\#]$
 (f est ignoré $\rightarrow$ utilisation d'une **affectation faible** qui accumule les valeurs)
- si $X^\#(V) = X^\#(A^\#) = [a, b]$:
 $S^\# \llbracket V \leq c \rrbracket X^\# \stackrel{\text{def}}{=} X^\# [V \mapsto [a, \min(b, c)]]$ si $a \leq c$, $\perp$ sinon
 $S^\# \llbracket A[e] \leq c \rrbracket X^\# \stackrel{\text{def}}{=} X^\#$ si $a \leq c$, $\perp$ sinon
 (test de satisfiabilité, mais pas de raffinement de la valeur de $X^\#(A)$
 le cas $A[e] \leq A[f]$ est similaire)
- les autres opérations restent inchangées : $\cap^\#$, $\cup^\#$, ...

Exemple d'analyse sur les intervalles

Exemple : sous-séquence croissante

```
p[0] ← 0; B[0] ← A[0];  
i ← 1; k ← 1;  
while i < N do  
    if A[i] > B[k - 1] then  
        B[k] ← A[i];  
        p[k] ← i;  
        k ← k + 1;  
    i ← i + 1  
done
```

Résultat :

En supposant $N \in [N_\ell, N_h]$, $\forall x: A[x] \in [A_\ell, A_h]$, on trouve :

- $\forall x: p[x] \in [0, N_h - 1]$
- $\forall x: B[x] \in [\min(0, A_\ell), \max(0, A_h)]$

Abstractions relationnelles

Ajout et suppression de variables

Sémantique concrète :

L'ensemble $\mathbb{V}$ des variables varie lors de l'exécution du programme
(e.g., variables locales)

maintenant : $\mathcal{E} \stackrel{\text{def}}{=} \bigcup_{\mathbb{V} \text{ fini}} \mathbb{V} \rightarrow \mathbb{Z}$

- $S[\text{add } V] R \stackrel{\text{def}}{=} \{ \rho[V \mapsto v] \mid \rho \in R, v \in \mathbb{Z} \}$
(ajout d'une variable non initialisée)
- $S[\text{del } V] R \stackrel{\text{def}}{=} \{ \rho|_{\text{dom}(\rho) \setminus \{V\}} \mid \rho \in R \}$
(suppression d'une variable)

Sémantique abstraite :

$\mathcal{E}^\# \stackrel{\text{def}}{=} \bigcup_{\mathbb{V} \text{ fini}} \mathcal{E}_{\mathbb{V}}^\#$

(un domaine abstrait sur $|\mathbb{V}|$ dimensions pour chaque $\mathbb{V}$ possible, e.g. : $\mathcal{E}_{\mathbb{V}}^\# = \text{polyèdres de } \mathbb{R}^{|\mathbb{V}|}$)

Exemple, dans les intervalles :

- $S^\#[\text{add } V] X^\# \stackrel{\text{def}}{=} X^\#[V \mapsto [-\infty, +\infty]]$
- $S^\#[\text{del } V] X^\# \stackrel{\text{def}}{=} X^\#|_{\text{dom}(X^\#) \setminus \{V\}}$

Duplication et repliage de variables

expand et fold : modélisent le repliage dynamique

$$S[\text{expand } V \rightarrow V'] R \stackrel{\text{def}}{=} \{ \rho[V' \mapsto v] \mid \rho \in R \wedge \rho[V \mapsto v] \in R \}$$

$$S[\text{fold } V \leftarrow V'] R \stackrel{\text{def}}{=} \{ \rho \mid \exists v: \rho[V' \mapsto v] \in R \vee \rho[V' \mapsto \rho(V), V \mapsto v] \in R \}$$

- **expand copie** une variable et ses contraintes

$(1 \leq V \leq X \implies 1 \leq V \leq X \wedge 1 \leq V' \leq X; \text{ mais } V = V' \text{ n'est pas vrai!})$

- **fold replie** V et V' dans V

$(1 \leq V \leq X \wedge 2 \leq V' \leq Y \implies 1 \leq V \leq X \vee 2 \leq V \leq Y)$

- **fold** est une **abstraction**, **expand** est la **concrétisation** associée :

$$\mathcal{P}(\mathbb{V} \rightarrow \mathbb{Z}) \xrightleftharpoons[S[\text{fold } V \leftarrow V']]{S[\text{expand } V \rightarrow V']} \mathcal{P}((\mathbb{V} \setminus \{V'\}) \rightarrow \mathbb{Z})$$

(on a une correspondance de Galois)

Expand et fold relationnel

Abstraction dans les polyèdres :

- **expand** peut être modélisé exactement par des **copies de contraintes** :

$$S^\# \llbracket \mathbf{expand} \ V_a \rightarrow V_b \rrbracket \{ \sum_i \alpha_{ij} V_i \geq \beta_j \} \stackrel{\text{def}}{=} \{ \sum_i \alpha_{ij} V_i \geq \beta_j \} \cup \{ \sum_{i \neq a} \alpha_{ij} V_i + \alpha_{aj} V_b \geq \beta_j \}$$

- **fold** peut être sur-approximé par une **copie faible** :

$$S^\# \llbracket \mathbf{fold} \ V \leftarrow V' \rrbracket X^\# \stackrel{\text{def}}{=} S^\# \llbracket \mathbf{del} \ V' \rrbracket (X^\# \cup^\# S^\# \llbracket V \leftarrow V' \rrbracket X^\#)$$

(l'affectation garde les anciennes valeurs, en plus des nouvelles, au lieu de les remplacer)

exemple : $0 \leq V \leq 3 \wedge 10 \leq V' \leq 13 \implies 0 \leq V \leq 13$
qui sur-approxime $0 \leq V \leq 3 \vee 10 \leq V \leq 13$

- $S^\# \llbracket \mathbf{add} \ V \rrbracket$ garde les contraintes inchangées
- $S^\# \llbracket \mathbf{del} \ V \rrbracket$ projette V

Abstraction relationnelle des tableaux

But : abstraire $\mathcal{P}(\mathcal{E})$ par des polyèdres sur $\mathbb{V}^\# \stackrel{\text{def}}{=} \mathbb{V} \cup \mathbb{A}$

Principe : utiliser *join* et *expand* en interne

Abstraction de l'affectation : $S^\# \llbracket A[f] \leftarrow e \rrbracket X^\#$

- un accès de tableau $A[expr]$ dans e est remplacé par une copie de $A^\#$ ce qui donne une nouvelle expression e' et un nouvel environnement $X_1^\#$

e.g., remplacer $B[expr]$ dans $X^\#$, par $B^{\#'}$ dans $X_1^\# \stackrel{\text{def}}{=} S^\# \llbracket \text{expand } B^\# \rightarrow B^{\#'} \rrbracket X^\#$

- création d'une copie $A^{\#'}$ de $A^\#$, pour contenir le résultat

$X_2^\# \stackrel{\text{def}}{=} S^\# \llbracket \text{expand } A^\# \rightarrow A^{\#'} \rrbracket X_1^\#$

- affectation de e' dans $A^{\#'}$

$X_3^\# \stackrel{\text{def}}{=} S^\# \llbracket A^{\#'} \leftarrow e' \rrbracket X_2^\#$

- repliage de $A^{\#'}$ dans $A^\#$

$X_4^\# \stackrel{\text{def}}{=} S^\# \llbracket \text{fold } A^\# \leftarrow A^{\#'} \rrbracket X_3^\#$

- suppression des copies de tableau :

$S^\# \llbracket \text{del } B^{\#'} \rrbracket X_4^\#$

$(S^\# \llbracket V \leftarrow e \rrbracket \text{ et } S^\# \llbracket c? \rrbracket \text{ se traitent de la manière similaire})$

Exemple d'analyse dans les polyèdres

Exemple : sous-séquence croissante

```
p[0] ← 0; B[0] ← A[0];  
i ← 1; k ← 1;  
while i < N do  
    if A[i] > B[k - 1] then  
        B[k] ← A[i];  
        p[k] ← i;  
        k ← k + 1;  
    i ← i + 1  
done
```

Résultats :

Avec l'hypothèse $\forall x: A[x] \in [A_\ell, A_h]$, on trouve :

- $\forall x: 0 \leq p[x] < N$
(plus fort que $\forall k: 0 \leq p[k] < N_h$)
- $\forall x: B[x] \in [\min(0, A_\ell), \max(0, A_h)]$
(Note : $B \leq A$ n'est pas vrai, car il serait interprété comme $\forall i, j: B[i] \leq A[j]$)